High-Level Data Design Document

Project 07 | Data Repositories, Writers, Versioning, Lineage & End-to-End Flow

GIthub - https://github.com/mahi397/ML-Sys-Ops-Project

1. System Scope and Assumptions

This document describes the data layer for a meeting-summarisation system hosted on a Chameleon VM. There is no lakehouse involved. Instead, the design relies on three concrete stores that are already available in the environment:

- **Block storage** is mounted at `/mnt/block` and used for all local staging and runtime working files.
- **Object storage** bucket `objstore-proj07` holds the raw AMI corpus and all durable output artifacts.
- **rclone** is pre-configured on the VM for transfers between block and object storage.
- **Docker** is installed.

2. Repository Inventory

Four repositories together cover all data needs for the project.

Repository	Primary Role	Versioning Mechanism
`proj07_sql_db`	Canonical relational state - meetings, utterances, segments, recaps, feedback, lineage	Column-level version fields + dataset_versions table
`objstore-proj07`	Durable artifact store - raw corpus, processed outputs, training datasets, request/response logs	Explicit vN path prefixes (e.g. datasets/roberta_stage1/v2/)

/mnt/block	Local execution staging - working copies, logs, runtime intermediates	Mirrors object-store snapshots; log rotation by filename
GitHub repo	Source control - schemas, runtime bundles, env templates, scripts, documentation	Git commits and semantic tags

3. PostgreSQL Data Design

3.1 Purpose

proj07_sql_db is the single source of truth for structured meeting state. Every runtime reads or writes through here for anything that needs querying, joining, or auditability.

3.2 Explicit Data Schema

The schema is deliberately flat and meeting-centric. All tables hang off a meetings primary key.

Core Meeting Schema

- meetings (meeting_id TEXT PK, source_type TEXT, source_name TEXT, started_at TIMESTAMPTZ, ended_at TIMESTAMPTZ, raw_folder_prefix TEXT, dataset_version INTEGER, dataset_split TEXT)
- meeting_speakers (meeting_speaker_id BIGSERIAL PK, meeting_id TEXT FK, user_id BIGINT FK, speaker_label TEXT, display_name TEXT, role TEXT)
- meeting_artifacts (artifact_id BIGSERIAL PK, meeting_id TEXT FK, artifact_type TEXT, object_key TEXT, content_type TEXT, artifact_version INTEGER, created_at TIMESTAMPTZ)

Transcript & Boundary Schema

- utterances (utterance_id BIGSERIAL PK, meeting_id TEXT FK, meeting_speaker_id BIGINT FK, utterance_index INTEGER, start_time_sec DOUBLE, end_time_sec DOUBLE, raw_text TEXT, clean_text TEXT, source_segment_id TEXT)
- utterance_transitions (transition_id BIGSERIAL PK, meeting_id TEXT FK, left_utterance_id BIGINT FK, right_utterance_id BIGINT FK, transition_index INTEGER, gold_boundary_label BOOLEAN, pred_boundary_prob DOUBLE, pred_boundary_label BOOLEAN)

-

Recap & Lineage Schema

- topic_segments (topic_segment_id BIGSERIAL PK, meeting_id TEXT FK, segment_type TEXT, segment_index INTEGER, start_utterance_id BIGINT FK, end_utterance_id BIGINT FK, start_time_sec DOUBLE, end_time_sec DOUBLE, topic_label TEXT)
- summaries (summary_id BIGSERIAL PK, meeting_id TEXT FK, summary_type TEXT, summary_object_key TEXT, created_by_user_id BIGINT FK, version INTEGER, created_at TIMESTAMPTZ)
- segment_summaries (segment_summary_id BIGSERIAL PK, meeting_id TEXT FK, topic_segment_id BIGINT FK, summary_id BIGINT FK, segment_index INTEGER, topic_label TEXT, summary_bullets JSONB, status TEXT, model_name TEXT, model_version TEXT, prompt_version TEXT, created_at TIMESTAMPTZ)
- feedback_events (feedback_event_id BIGSERIAL PK, meeting_id TEXT FK, summary_id BIGINT FK, segment_summary_id BIGINT FK, event_type TEXT, event_source TEXT, before_payload JSONB, after_payload JSONB, created_by_user_id BIGINT FK, created_at TIMESTAMPTZ)
- dataset_versions (dataset_version_id BIGSERIAL PK, dataset_name TEXT, stage TEXT, source_type TEXT, object_key TEXT, manifest_json JSONB, created_at TIMESTAMPTZ)

4. Object Storage Data Design

4.1 Purpose

objstore-proj07 holds everything too large or too raw for the relational store, the original AMI corpus, processed transcripts, training datasets, and all endpoint request/response payloads. It is write-once by convention; new versions get new paths rather than overwriting old ones.

4.2 Path Layout

- ami_public_manual_1.6.2/ (corpusResources/, ontologies/, words/, segments/, topics/, abstractive/)
- processed/ami/v1/
  - transcripts/{meeting_id}.json
  - summaries/{meeting_id}.json
  - manifests/{meeting_id}.json
- datasets/roberta_stage1/vN/ (train.jsonl, val.jsonl, test.jsonl, split_info.json, examples.json, manifest.json)
- datasets/roberta_stage1_feedback_pool/vN/ (feedback_examples.jsonl, meeting_ids.json, examples.json, manifest.json)
- production/jitsi/
  - raw_transcripts/{meeting_id}/vN_transcript_<ts>_<uuid>.txt
  - parsed_transcripts/{meeting_id}/vN.json

- production/inference_requests/
  - stage1/{meeting_id}/ (requests.jsonl, requests.json, model_utterances.json, manifest.json)
  - stage2/{meeting_id}/ (inputs.jsonl, inputs.json)
- production/inference_responses/
  - stage1/{meeting_id}/ (responses.jsonl, responses.json)
  - stage2/{meeting_id}/ (responses.jsonl, responses.json)
- production/reconstructed_segments/{meeting_id}/vN.json
- production/recaps/generated/{meeting_id}/vN.json
- production/recaps/edited/{meeting_id}/vN.json
- production/feedback_events/{meeting_id}/events_vN.jsonl

5. Block Storage Design

5.1 Purpose

The VM's attached block volume (/mnt/block) is the local working area. Runtimes stage data here before uploading to object storage, and logs live here permanently. Nothing in /mnt/block is the canonical record that role belongs to PostgreSQL and object storage but losing it would require re-running jobs to recover the intermediate state.

5.2 Directory Layout

- /mnt/block/postgres-data/
- /mnt/block/ingest_logs/
- /mnt/block/roberta_stage1/vN/
- /mnt/block/staging/
  - current_job/raw/
  - current_job/processed/
  - feedback_loop/feedback_events/{meeting_id}/events_vN.jsonl
  - feedback_loop/edited_recaps/{meeting_id}/vN.json
  - feedback_loop/datasets/roberta_stage1_feedback_pool/vN/
- /mnt/block/user-behaviour/
  - Transcripts/
  - parsed_transcripts/{meeting_id}/parsed_transcript.json
  - online_inference/stage1/{meeting_id}/vN/
  - online_inference/stage2/{meeting_id}/vN/
  - inference_responses/stage1/{meeting_id}/vN/
  - inference_responses/stage2/{meeting_id}/vN/
  - reconstructed_segments/{meeting_id}/vN.json
  - recaps/generated/{meeting_id}/vN.json

6. Writers, Update Timing, and Responsibilities

Runtime	When It Runs	What It Writes
external_data_training_runtime	Once at project start, or on full rebuild	Canonical AMI rows in PostgreSQL; processed/ami/v1/* and roberta_stage1/v1 & v2 in object storage
endpoint_replay_runtime	To hit hypothetical end points	Synthetic request/response artifacts and recap outputs to object storage
online_inference_workflow_runtime	Whenever a new production transcript arrives	Jitsi meeting rows, runtime artifacts, generated and corrected recaps, feedback events in both PostgreSQL and object storage
retraining_dataset_runtime	When enough corrected meetings have accumulated	Rolling feedback-pool artifacts, a new roberta_stage1/vN snapshot, dataset_versions rows, and dataset_version / dataset_split stamps in meetings

7. Versioning and Lineage Strategy

7.1 Artifact Versioning

Object-storage paths use explicit `vN` segments so multiple versions of the same dataset can coexist without any ambiguity about which is current. Examples:

- datasets/roberta_stage1/v1/, /v2/, /vN/
- production/jitsi/parsed_transcripts/{meeting_id}/v1.json
- production/feedback_events/{meeting_id}/events_v1.jsonl

7.2 Row-Level Versioning

PostgreSQL carries version state at the row level:

- `meetings.dataset_version` and `meetings.dataset_split` record released-dataset membership. NULL means a meeting has not yet been promoted into any training split.
- `meeting_artifacts.artifact_version` tracks revisions to individual runtime artifacts.
- `summaries.version` tracks recap revisions independently of the meeting row.

7.3 Dataset Lineage

The `dataset_versions` table is the lineage registry. Every released snapshot gets a row recording its dataset_name, stage, source_type, the object-store manifest path, and the full manifest_json. Any later investigation can reconstruct exactly which meetings were in which split, when, and why.

7.4 Transformation Traceability

Raw and parsed artifacts are both stored durably and linked through `meeting_artifacts`, so the chain from raw transcript to final model input can be reconstructed at any point. Processed AMI artifacts are versioned before Stage 1 dataset compilation, meaning the dataset build step always starts from a known, fixed input.

7.5 Leakage Prevention

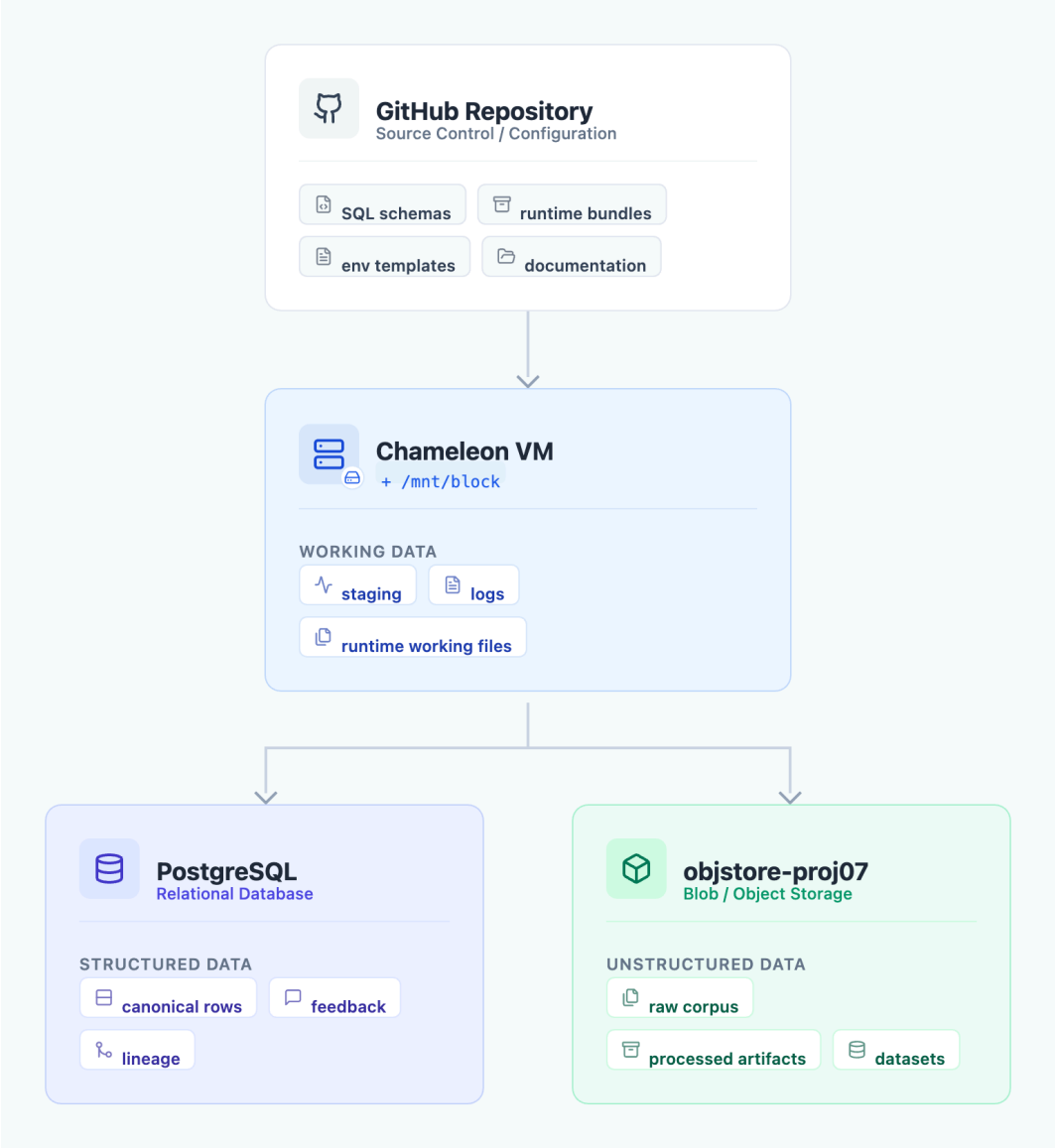
This is the most operationally sensitive part of the design. The rules are:

- All AMI meetings are assigned explicit train / val / test membership when roberta_stage1/v1 is created. They never move between splits.
- Jitsi meetings stay at dataset_version = NULL until retraining dataset creation pipeline deliberately promotes them. It only discovers meetings with NULL version and available structural feedback, so nothing can silently re-enter a subsequent evaluation slice.
- When a new rolling snapshot is built, the prior dataset version is folded into the train split. Only newly eligible Jitsi meetings are assigned to val and test. Once stamped, a meeting cannot be re-assigned.

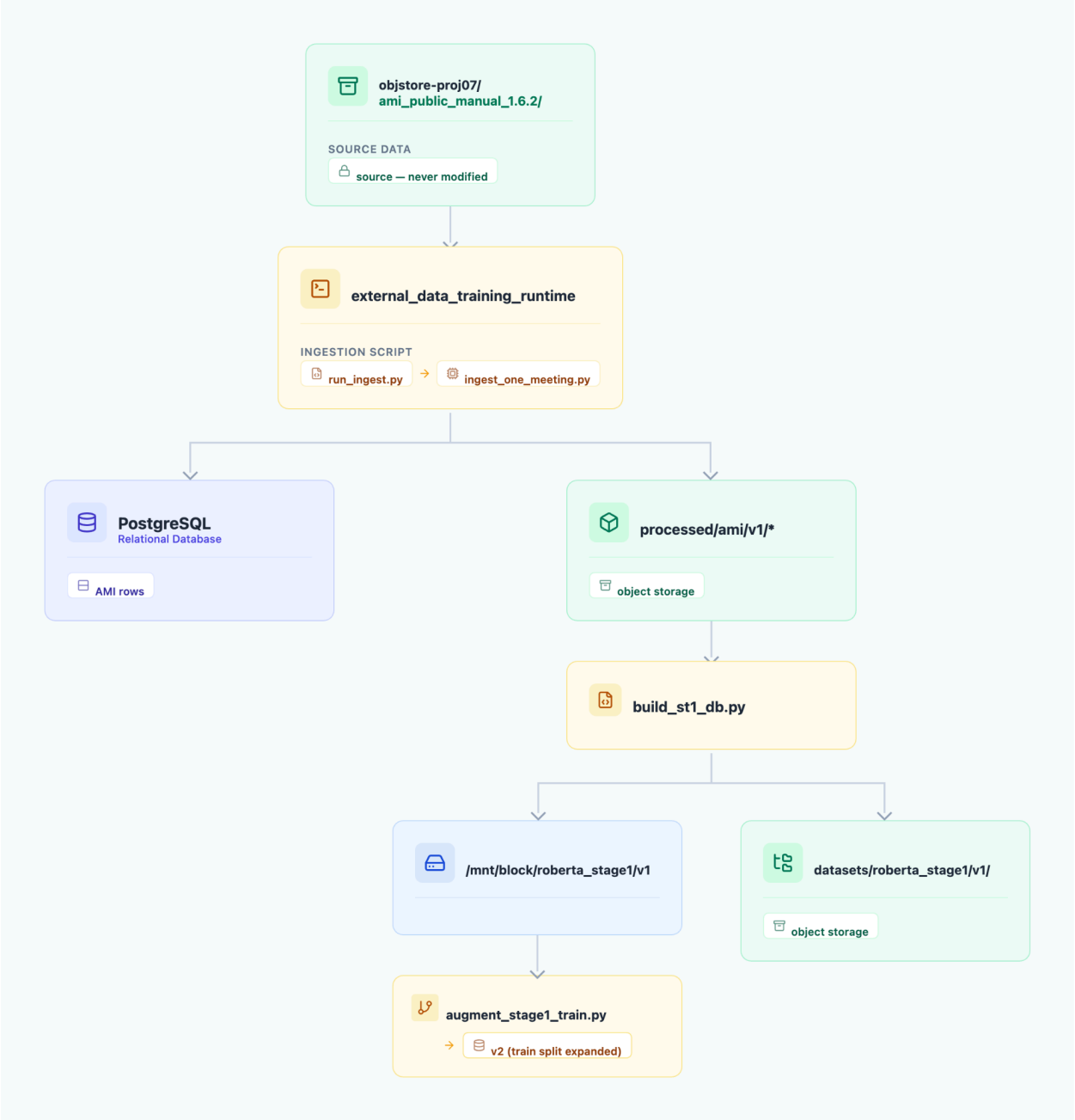
8. Data-Flow Diagrams

Storage Architecture

The diagram below shows how the four repositories relate to each other and to the VM environment.



Historical Training Data Flow



Production to Retraining Flow